Security in Software Applications

Testing

SAPIENZA
UNIVERSITÀ DI ROMA

Daniele Friolo

friolo@di.uniroma1.it

https://danielefriolo.github.io

Testing

- Testing is a costly part of development efforts
- How are these testing efforts used in secure programming?
  - Scenario testing
  - Specification testing
  - In-line testing
  - Unit testing
  - Integration testing
  - Human interaction testing
  - Final testing
  - Acceptance testing

Scenario Testing

- Used to ensure that requirements capture is complete and consistent
- Validation effort
- Example: Abuse and Misuse cases (UML)
  - If I were a hacker, what would I try?
    - Identify trust relationships and attack them
    - Which resources are open to me? How can they be abused?
    - Are there insecure defaults or things that could easily be misconfigured or badly coded?

Question

- Scenario testing requires from the tester:

a) A solid understanding of the programming language

b) An imagination and inventiveness for possible risks

c) Experience being a cracker and "owning" hosts

Specification Testing

- Prove the completeness and correctness of the specification
  - Formal proof
  - Symbolic execution
- This is usually done only for high-assurance efforts
- Many networking protocols lacked specification testing for security and are riddled with vulnerabilities
  - Making your own (secure) communication protocols is not trivial!

In-line Testing

- Execution testing
  - e.g., ASSERT
- Verify that the internal state of the software is:
  - allowed (by the security policies)
  - expected
  - self-consistent
- Verify invariants and rules
  - "Manual" model checking

Disadvantages of Inline Testing

- Usually turned off in deployed applications (e.g., Java)
  - Verifying invariants can incur significant overhead
  - Errors may not be user-friendly
  - Data integrity could be compromised
- Need to cover as much code as possible
  - *"No run == no bug"* (D. Engler)
  - Manual placement of ASSERTs and tests may miss violations
- Some rules and assumptions difficult to test
- Will not help if execution is transfered successfully to malicious code

Question

- Inline testing is useful for security because it:
  a) Allows the verification of invariants
  b) Is usually turned off in the final product
  c) Happens earlier than other forms of testing

- Comment: c) is indirectly true in the sense that it is better to catch bugs early. However the primary reason is a).

Unit Testing

- Local testing of functions, etc…
- Good time to test against buffer overflows and other low-level coding mistakes

Human Interaction Testing

- Does the UI lead users to make the correct security decisions?
- Does the UI properly and effectively convey security information to the user?
- Are error messages informative and pertinent?
  - Click-through syndrome
- Resilience to User Interface Abuse
  - Phishing, social engineering
  - Colored text so human-readable message is different from machine interpretation
  - Obscured links, file names

Integration Testing

- Interfaces, linking & loading
- Test for discrepancies in assumptions and security models
  - who was supposed to do authentication and access control?
- Security testing
  - Ensure that all operations that should be denied, are denied
  - Verify that partial accesses can only access what they should

Acceptance Testing

- Customer validation step (ET - external test in beta)
- Should include tests for likely threats
  - Also, flaw-hypothesis testing
    - Make hypotheses as to where flaws are likely and try to expose them
  - Customer may not possess necessary skills!
    - Customer trusts reviewer
      - Reviewer responsibility to find flaws
      - Is the reviewer hamstrung by an "obligation" to be nice to the vendor providing the software for review?
    - Supposedly well-intentioned researchers or hackers might do so "for the public's sake"

Final Testing

- Test cases run against entire project
- Fault injection
  - Random input
    - What happens if input is nonsense?
      - Denial of service (crashes), etc…
        - indicate possible compromises (buffer overflows)
    - Bug isolation can be difficult
      - Binary search:
        - Bug is in X sequence of inputs, but not in first half nor second half
        - Depends on internal state of project
        - State of system unknown
  - Grammar-generated input

Question (and Sample Answers)

- Describe limitations of using random input for final (security) testing.
  - Not all execution paths are tested
  - Pseudo-random input is not completely random
  - It may be difficult to identify and locate the bug, because random input long before a crash may have produced a special internal state of the software

Question

- In black box testing, the tester has no knowledge of the internal workings of the compiled, final artifact.
  Which kind of testing CAN'T you do in black box testing?

a) Final testing
b) Acceptance testing
c) Inline testing